

Table Of Contents

I.INTRODUCTION	2
II. PRIMARY DATA TYPES AND OS ALLOCATION	2
A. Data Type Sizes Across Architectures	2
B. Practical Example: Observing Data Type Sizes	3
III. OS MEMORY SEGMENTS	5
A. Memory Segment Overview	5
B. Memory Layout Visualization.....	5
IV. PRACTICAL CODE EXAMPLES	5
A. Observing Variable Addresses	5
B. Pointer and Memory Allocation	7
V. LABORATORY EXERCISES	8
A. Lab 1: Data Type Size Exploration.....	8
B. Lab 2: Memory Segment Address Mapping.....	9
VI. SUMMARY AND KEY TAKEAWAYS	10
Knowledge Check Questions:	10

I.INTRODUCTION

Data types form the foundation of programming and operating system interaction. A data type is a contract between the programmer, compiler, and operating system that specifies:

1. The number of bytes to allocate in memory
2. How to interpret those bytes (integer, character, floating-point)
3. What operations are valid on that data

When a programmer declares a variable, the OS must allocate physical or virtual memory to store that data. Understanding this relationship is important for writing efficient, secure, and portable code.

II. PRIMARY DATA TYPES AND OS ALLOCATION

A. Data Type Sizes Across Architectures

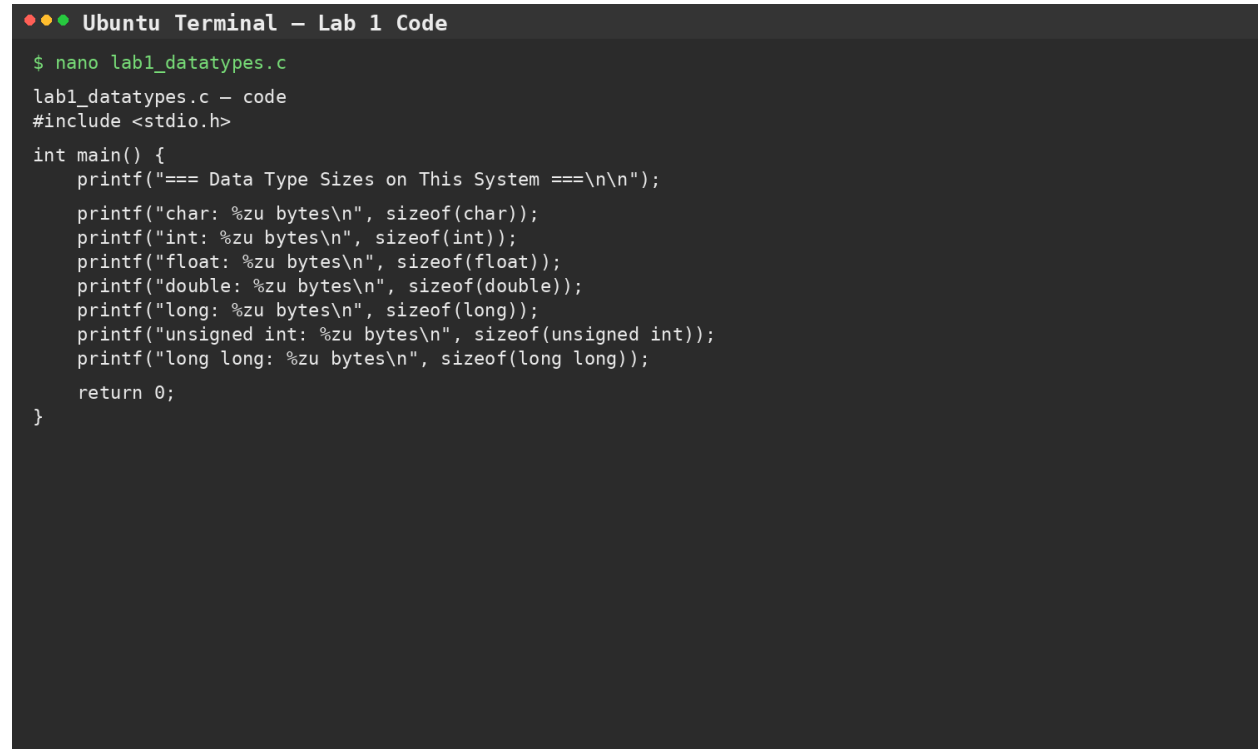
Table I shows how data type sizes vary across different system architectures, directly impacting OS memory allocation.

TABLE I

Data Type	16-bit	32-bit	64-bit	OS Impact
char	1 byte	1 byte	1 byte	Consistent across architectures
short	2 bytes	2 bytes	2 bytes	Memory-efficient counters
int	2 bytes	4 bytes	4 bytes	Affects pointer arithmetic
long	4 bytes	4 bytes	8 bytes	Critical for 64-bit addressing
long long	8 bytes	8 bytes	8 bytes	Large file offsets
float	4 bytes	4 bytes	4 bytes	IEEE 754 standard
double	8 bytes	8 bytes	8 bytes	Precision calculations
pointer	2 bytes	4 bytes	8 bytes	Address space limit

B. Practical Example: Observing Data Type Sizes

The program was created to check the memory size of different data types on the system.



```

$ nano lab1_datatypes.c
lab1_datatypes.c - code
#include <stdio.h>

int main() {
    printf("=== Data Type Sizes on This System ===\n\n");
    printf("char: %zu bytes\n", sizeof(char));
    printf("int: %zu bytes\n", sizeof(int));
    printf("float: %zu bytes\n", sizeof(float));
    printf("double: %zu bytes\n", sizeof(double));
    printf("long: %zu bytes\n", sizeof(long));
    printf("unsigned int: %zu bytes\n", sizeof(unsigned int));
    printf("long long: %zu bytes\n", sizeof(long long));
    return 0;
}

```

The program was compiled using GCC. The successful compilation shows that there were no errors in the program.

```
Ubuntu Terminal - Lab 1 Compile
$ gcc lab1_datatypes.c -o lab1_datatypes
$ echo $?
0
Compilation completed successfully with no errors.
```

The program was then executed to display the size of each data type.

```
Ubuntu Terminal - Lab 1 Output
$ ./lab1_datatypes
=== Data Type Sizes on This System ===
char: 1 bytes
int: 4 bytes
float: 4 bytes
double: 8 bytes
long: 8 bytes
unsigned int: 4 bytes
long long: 8 bytes
```

The output was compared with the values given in Table I.

III. OS MEMORY SEGMENTS

The operating system creates a process address space a virtual view of memory divided into different segments. Each segment serves a specific purpose and has different characteristics.

A. Memory Segment Overview

TABLE II

PROCESS MEMORY SEGMENTS AND THEIR PURPOSES

Segment	Purpose	Grows	Managed By
Text	Machine code (instructions)	Fixed	Compiler/Loader
Data	Initialized global/static variables	Fixed	Compiler/OS
BSS	Uninitialized global/static variables	Fixed	OS
Heap	Dynamically allocated memory	Upward	Programmer + OS
Stack	Local variables, function calls	Downward	Compiler + OS

B. Memory Layout Visualization

The process memory is divided into different sections. The Stack is generally located at higher virtual addresses, while the Heap is located in the middle area and the Data/BSS sections are located at lower addresses.

Key Observation: Stack addresses are generally high, Heap addresses are in the middle range, and Data/BSS addresses are generally lower.

IV. PRACTICAL CODE EXAMPLES

A. Observing Variable Addresses

The program was used to display the memory addresses of variables stored in different memory segments.

Ubuntu Terminal – Lab 2 Code

```
$ nano lab2_memory_segments.c
lab2_memory_segments.c – code
#include <stdio.h>
#include <stdlib.h>

int global_var = 150; // Data segment
int bss_var; // BSS segment

int main() {
    int local_var = 30; // Stack
    static int local_static = 20; // Data segment

    int *heap_var = (int *)malloc(sizeof(int)); // Heap
    *heap_var = 500;

    printf(".....Variable Addresses.....\n\n");
    printf("global_var: %p (Data)\n", (void *)&global_var);
    printf("bss_var: %p (BSS)\n", (void *)&bss_var);
    printf("local_var: %p (Stack)\n", (void *)&local_var);
    printf("local_static: %p (Data/BSS)\n", (void *)&local_static);
    printf("heap_var ptr: %p (Stack - pointer)\n", (void *)&heap_var);
    printf("*heap_var: %p (Heap - data)\n", (void *)heap_var);

    printf("\nStack-Heap address difference: %ld bytes\n",
        (long)((char *)&local_var - (char *)heap_var));

    free(heap_var);
    return 0;
}
```

The program was compiled successfully using GCC.

Ubuntu Terminal – Lab 2 Compile

```
$ gcc lab2_memory_segments.c -o lab2_memory_segments
$ echo $?
0
Compilation completed successfully with no errors.
```

The program was then executed to display the addresses of the variables.

```

●●● Ubuntu Terminal – Lab 2 Output
$ ./lab2_memory_segments
.....Variable Addresses.....
global_var: 0x555555558010 (Data)
bss_var:    0x555555558014 (BSS)
local_var:  0x7fffffffdc4c (Stack)
local_static: 0x555555558018 (Data/BSS)
heap_var ptr: 0x7fffffffdc50 (Stack - pointer)
*heap_var:   0x5555555592a0 (Heap - data)
Stack-Heap address difference: 1044940 bytes

```

The output shows that different variables are stored in different memory areas such as Data, BSS, Stack, and Heap.

B. Pointer and Memory Allocation

The program demonstrates how a pointer can be stored on the Stack while pointing to memory allocated on the Heap.

```

●●● Ubuntu Terminal – Lab 4 Commands
$ gcc lab1_datatypes.c -o lab1_datatypes
$ ./lab1_datatypes
$ gcc lab2_memory_segments.c -o lab2_memory_segments
$ ./lab2_memory_segments
Lab 4 command sequence completed.
```

The program was compiled and executed to display the pointer address, heap address, and stored value.

The output shows the difference between the address of the pointer and the address of the data stored in the Heap.

V. LABORATORY EXERCISES

A. Lab 1: Data Type Size Exploration

Objective

Use `sizeof()` to observe how much memory each data type uses on the system.

Tasks

Create a C program that prints the size of `char`, `int`, `float`, `double`, `long`, `unsigned int`, and `long long`.

Compile and run the program on the system.

Compare the output with Table I.

Answer: Why might `sizeof(long)` differ between 32-bit and 64-bit systems?

Result

The program successfully displayed the memory size of the selected data types. The results were compared with the expected values for the system architecture.

Answer

sizeof(long) may differ between 32-bit and 64-bit systems because different system architectures use different data models. On many 32-bit systems, long is 4 bytes, while on many 64-bit Linux systems it is commonly 8 bytes.

B. Lab 2: Memory Segment Address Mapping

Objective

Observe where different variables live in memory and validate the OS memory layout.

Tasks

Write a program that declares variables in each segment:

Global initialized variable (Data segment)

Global uninitialized variable (BSS segment)

Local variable (Stack segment)

Dynamically allocated variable (Heap segment)

Print the address of each variable using %p

Identify which segment has the highest addresses

Identify which segment has the lowest addresses

Calculate the address difference between stack and heap

Result

The program successfully displayed the memory addresses of variables from different memory segments.

The Stack generally had the highest addresses, while the Data/BSS regions had lower addresses. The Heap was located between these regions in the observed process layout.

Address Difference

The address difference between the Stack and Heap can be calculated using:

Address Difference = |Stack Address – Heap Address|

The actual difference depends on the memory addresses produced by the system when the program is executed.

VI. SUMMARY AND KEY TAKEAWAYS

- Data types tell the computer what kind of data we are storing and how much memory is needed for it.
- Memory is divided into different sections: Text stores program code, Data/BSS stores global and static variables, Heap is used for dynamic memory, and Stack stores local variables and function information.
- The OS manages it all—tracking, allocating, protecting, and virtualizing memory.
- Stack is automatic; heap is manual—choose based on lifetime and size needs.
- Virtual memory provides isolation—each process has its own address space.
- Pointers bridge stack and heap—the pointer variable lives on stack, but points to heap memory.

Knowledge Check Questions:

1) What is a data type in the context of programming and operating systems?

A data type tells the computer what kind of data a variable store, such as an integer, character, or floating-point number. It also helps determine how much memory is needed and how the stored data should be interpreted.

2) Name the five main memory segments in a process memory layout.

The five main memory segments are:

1. **Text (Code) Segment** – stores the program's instructions.
2. **Data Segment** – stores initialized global and static variables.
3. **BSS Segment** – stores uninitialized or zero-initialized global and static variables.
4. **Heap** – used for dynamically allocated memory.
5. **Stack** – stores local variables, function parameters, and function call information.

3) What is the difference between the Data segment and the BSS segment?

The Data segment stores global and static variables that have an initial value, such as `int x = 10;`. The BSS segment stores global and static variables that are uninitialized or initialized to zero, such as `int x;`.

4) Why does the OS place the stack at high virtual addresses and the text segment at low addresses?

The stack is placed at high virtual addresses and usually grows downward, while the heap grows upward from lower addresses. This arrangement leaves a large area of virtual address space between them, allowing both the stack and heap to grow without immediately colliding.

5) What happens to stack memory when a function returns? Who reclaims it?

When a function returns, its stack frame is automatically removed, making that memory available for future function calls. The compiler-generated code and CPU's stack mechanism handle this; the programmer does not normally need to manually free stack memory.

6) How does the OS know which memory belongs to which process?

The OS gives each process its own virtual address space and keeps track of how virtual addresses map to physical memory. Hardware mechanisms such as page tables ensure that a process can access only the memory it is allowed to access, preventing one process from accidentally accessing another process's memory.